

Lista Duplamente Encadeada

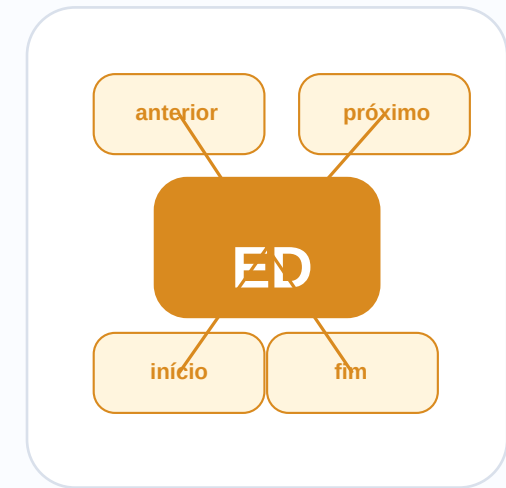
Explicação visual sobre nós com anterior e próximo, percurso duplo e manutenção de ponteiros.

anterior

próximo

início

fim



Ideia principal

Cada nó conhece o anterior e o próximo, permitindo percorrer a lista nos dois sentidos.

Quando usar

Quando navegação para frente e para trás, inserções e remoções são importantes.

Cuidado

Cada alteração mexe em mais ponteiros que uma lista simples.

Anterior + valor + próximo

O nó guarda três partes: referência anterior, valor armazenado e referência para o próximo nó.



Anterior

Permite voltar para o nó que vem antes.

Próximo

Permite avançar para o nó que vem depois.

A lista anda nos dois sentidos

Com início e fim, é possível percorrer da esquerda para a direita ou da direita para a esquerda.

Do início ao fim

Comece no primeiro nó e siga sempre pelo ponteiro próximo até chegar em null.

Do fim ao início

Comece no último nó e siga sempre pelo ponteiro anterior até chegar em null.

Vantagem

É útil em históricos, editores, playlists, navegadores e telas com avançar/voltar.

Custo

Ainda pode haver busca $O(n)$; a vantagem é a navegação reversa quando já existe uma referência.

Como inserir um novo nó

A regra é atualizar o novo nó e também os vizinhos afetados.

No início

1

novo.Proximo aponta para o antigo primeiro; o antigo primeiro.Anterior aponta para novo.

No fim

2

novo.Anterior aponta para o antigo último; o antigo último.Proximo aponta para novo.

No meio

3

novo liga anterior e próximo; depois os dois vizinhos passam a apontar para novo.

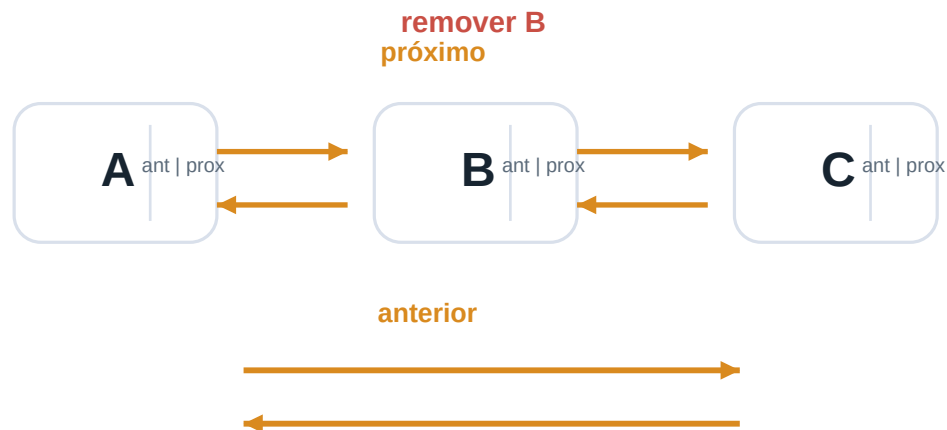
Lista vazia

4

início e fim apontam para o mesmo novo nó.

Como remover sem quebrar a lista

Para remover um nó, conecte o anterior ao próximo e o próximo ao anterior.



Depois da remoção

A.Proximo aponta para C e
C.Anterior aponta para A.

Casos de borda

Se remover o primeiro ou último
nó, atualize início ou fim.

Modelo de nó duplo em C#

Uma implementação simples com referências para anterior e próximo.

NoDuplo

```
public class NoDuplo
{
    public int Valor;
    public NoDuplo Anterior;
    public NoDuplo Proximo;

    public NoDuplo(int valor)
    {
        Valor = valor;
        Anterior = null;
        Proximo = null;
    }
}
```

Valor

Guarda o dado armazenado.

Anterior e Proximo

Permitem voltar e avançar entre os nós.

Vantagens e cuidados

A lista dupla é flexível, mas exige atenção porque cada alteração mexe em mais referências.

Vantagens

Percorre para frente e para trás. Facilita remoções quando o nó já é conhecido.

Cuidados

Usa mais memória que a lista simples por guardar duas referências em cada nó.

Erros comuns

Atualizar só um lado da ligação, esquecer null nos extremos ou não ajustar início e fim.

Como explicar

Mostre sempre os ponteiros antes e depois da operação.